PASADHI Fireworks Evaluation under Serverless Load Updated Full White Paper and Evidence Audit

PASADHI contributors | Full White Paper v1.1 | 28 October 2026

Abstract—Version 1.1 extends the PASADHI Fireworks evidence audit with nine serverless-load exports containing 22 phases and 11,101 logical requests. Combined with the historical cohort, the supplied corpus contains 37 run/phase identifiers and 11,221 unique request records, including 9,026 nonempty live outputs and 50 simulated records. New phases yield 9,007 successes and 2,094 errors: 1,841 HTTP 429 details and 253 empty/unsupported outputs. MiniMax and GPT-OSS each complete 500 of 500 requests at requested label 500, with approximately 38.55 and 41.77 successful outputs/s; at label 1,000, error fractions rise to 32.6% and 36.4%. A two-route phase passes an error-only 5% criterion at label 1,000 but has successful-request p95 of 80.48 s. Newer records retain error timing and verification keys, although all 3,500 finish-reason values remain null and a reached-provider flag is partly heuristic. Every added request contains only 848 characters, so no new long-context capability is demonstrated. We preserve v1.0 findings, recompute phase metrics, distinguish logical requests from retries and attempt rates from useful throughput, and delimit the evidence required for future production or training claims.

Keywords—PASADHI; Fireworks; inference evaluation; long-context validation; reproducibility; simulation

I. Introduction

PASADHI investigates how office compute and hosted model endpoints can support practical AI workflows. A useful evaluation must establish which backend actually executed, what work was performed, and whether the reported latency or throughput corresponds to useful output. This paper focuses on the result folder supplied by the project owner, rather than extending earlier Docker GEMM experiments or initiating new provider calls.

The immediate application is text cleanup: correcting noisy English and Thai office notes while preserving names, deadlines, order identifiers, quantities and amounts. A short exact-response probe accompanies those examples. This is a plausible integration workload, but the retained records are small and do not include an independently labelled quality benchmark. A nonempty response is therefore treated as transport/application success, not proof that the content is correct.

The central finding is a separation of evidence. Some requests reached deployed models and returned text. A second family was configured as live but stopped at a local input check. A third family generated synthetic records. Combining those families into one performance score would obscure both the useful integration evidence and the failed long-context measurement design. We retain the failures and simulation results as engineering evidence, with their actual scope stated throughout.

II. Evidence Corpus and Provenance

The complete source is test-result/firework/ [1], [7]. Its original _work directory contains 15 historical runs, and three root comparison exports duplicate 20 of those request rows. Nine additional max-serverless root exports contain 22 new phase/run identifiers and 11,101 independent logical request records. Canonical identity is the run_id/sequence pair; duplicate views are excluded. Sections describing the original comparison, long-context validation and simulation explicitly retain their historical scope.

TABLE 1. EVIDENCE CLASSES AFTER DEDUPLICATION

Class	Runs	Requests	Observed outcome
Deployment comparison	3	20	19 model texts; 1 error
Live-configured context/load	6	50	50 local validation errors
Offline simulation	6	50	50 synthetic records

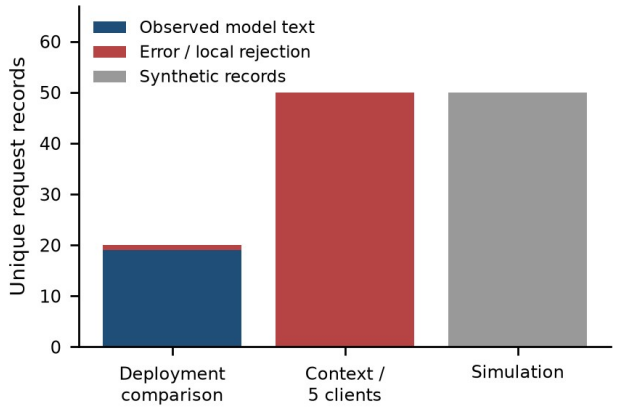


Fig. 1. Historical v1.0 cohort only: 120 canonical records after removing duplicate exports. The 11,101 new serverless requests are analyzed separately in the updated sections.

Every historical v1.0 request row was matched to the request events in its run log. Run counts, success/error counts, successful-token totals, per-cell means and nearest-rank percentiles were recomputed and compared with stored summaries. Sequence identifiers were unique within each run. All three root exports matched the corresponding canonical datasets. SHA-256 fingerprints of supplied files are retained in the derived analysis manifest [2].

The v1.0 audit recorded source hashes for its interpretation of the historical validation and simulation paths [3], [4]. Version 1.1 records new hashes of the now-visible source, which contains changes to the prompt cap and error instrumentation. Neither hash set is a historical run-time Git commit. Phase schema and source differences

are analyzed below; current implementation details are not silently projected backward onto older records.

The comparison exports contain local start times without a timezone suffix. Their epoch timestamps agree with Asia/Bangkok conversion: the principal run starts at 14:45:19 on 28 October 2026, equivalent to 07:45:19 UTC. The paper date and human-readable run timeline use Bangkok time. Source filenames and epoch values are preserved unchanged in the analysis.

III. Experimental Protocol Recovered from the Records

A. Deployment comparison

The comparison targets identify accounts/fireworks/models/muse-glimmer-30b and accounts/fireworks/models/llama-v3p1-8b-instruct, each with a deployment suffix. We refer to them as Glimmer and Llama. The exports call these dedicated deployments, and the identifiers support that routing intent. This audit did not query the provider control plane, so replica counts, GPU type, placement, deployment readiness and isolation cannot be independently established.

TABLE 2. DEPLOYMENT COMPARISON RUNS

Run time	Requests/ model	C	Max output
14:40:47	1	1	128
14:44:54	1	1	1024
14:45:19	8	2	1024

The four-prompt set consists of a short PING instruction, noisy English notes, noisy Thai notes and a mixed-language purchase-order line. The principal run has two observations for each prompt for each model. In the two smoke runs, each model receives only one prompt: Glimmer receives English and Llama receives Thai. Those smoke results are consequently not same-input model comparisons.

The harness constructs jobs by server/model and submits them to one ThreadPoolExecutor. The concurrency value is a shared client-worker count, not an independently enforced per-model load. The job list places the Glimmer group before the Llama group. Prompt assignment follows the request sequence modulo four. In this trace the main run has sequence identifiers 1–8 for Glimmer and 9–16 for Llama, introducing an execution-order confound despite balanced prompt counts.

B. Context and five-client probes

Five nominal context labels are retained: 6K, 16K, 64K, 256K and 600K. Their actual input lengths are counted in characters from config.json, not tokenized with the target model. Ten requests are configured at each of the first four sizes and five at 600K. A further five-client step reuses the smallest long prompt for five requests. All use concurrency five and max_tokens=32; the corresponding simulated runs mirror their inputs and counts.

TABLE 3. NOMINAL CONTEXT VERSUS MEASURED INPUT SIZE

Label	Characters	Live attempts
6K	24064	10
16K	64027	10
64K	256000	10
256K	1024072	10
600K	2400012	5
Five-client step	24064	5

The target label for these live-configured steps is glm-5p3-flash. This is a different target from the Glimmer/Llama comparison. A filename such as fw-serverless or an honesty block marked live indicates configuration intent; it does not by itself establish successful network dispatch. The actual error records and validation path determine the execution classification used here.

IV. Metric Definitions and Audit Rules

Successful live latency is the backend elapsed time around request construction, HTTP execution, payload parsing and text extraction, as recorded in elapsed_ms and propagated to latency_ms. It includes network and provider scheduling effects but does not isolate them. It is not time to first token, decoding-only time, or end-user task completion time. Historical failed rows use latency_ms=0.0; these values remain unavailable measurements. Later-schema phases record positive failure durations and are analyzed separately. This schema evolution prevents pooling all error latencies as one uniform sample.

Nearest-rank percentiles are recomputed as the sorted value at index ceil(pn), using one-based indexing. For n=8, both p95 and p99 select the largest observed latency. Their equality is a consequence of sample size and estimator, not evidence that the production tail is stable. With n=1, every percentile collapses to the same observation; such rows are smoke checks rather than distribution estimates.

Two throughput definitions are required. Attempt completion rate is all completed records divided by elapsed run time. Successful completion rate is nonempty successful live outputs divided by that time. The stored totals.rps and throughput buckets count all finished records, including errors. Per-cell rps counts accepted outputs but uses the common run duration. Neither measure is a per-deployment saturation throughput estimate in this shared, ordered workload.

For rates in the historical comparison sections, elapsed time is ended_at minus started_at from the raw summary timestamps. Displayed elapsed_s is rounded to two decimal places and is not used to reconstruct sub-second rates. No error-duration distribution is imputed. Tokens are the provider usage fields retained on successful responses, and reasoning tokens are treated as a subset of completion tokens when those fields are present. New serverless phase rates use the retained rounded elapsed_s values because phase-level start/end timestamps are not supplied in those summaries.

V. Live Deployment Results

TABLE 4. PRINCIPAL RUN LATENCY WITH EIGHT SUCCESSES PER MODEL

Metric (ms)	Glimmer	Llama
Mean	2974.07	765.09
p50	2740.29	534.50
p95 / p99	5346.19	1273.98
Observed minimum	1239.49	400.10
Observed maximum	5346.19	1273.98

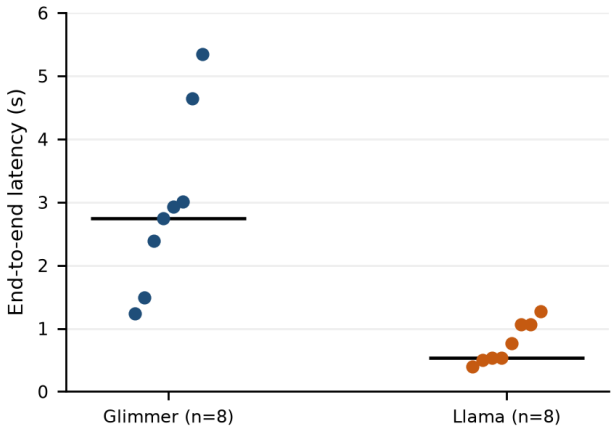


Fig. 2. Individual successful latencies in the principal live comparison. Each dot is one request; the black line is the nearest-rank median. Eight observations per model do not define a reliable production tail.

The principal run completes 16 of 16 requests with a stored elapsed time of 15.11 s. Glimmer has a mean latency 3.89 times Llama's in this trace. This is a descriptive ratio of sample means, not a general speedup guarantee: models differ in size, token accounting, reasoning behavior and output content, while deployment resources and request order are uncontrolled.

Both model cells have a reported rps of 0.53 because each returns eight responses within the same overall run window. That equality does not contradict the latency difference: it is imposed by the balanced request counts and common denominator. Aggregate attempt and successful completion rates are both approximately 1.06 requests/s for this run, with no observed errors.

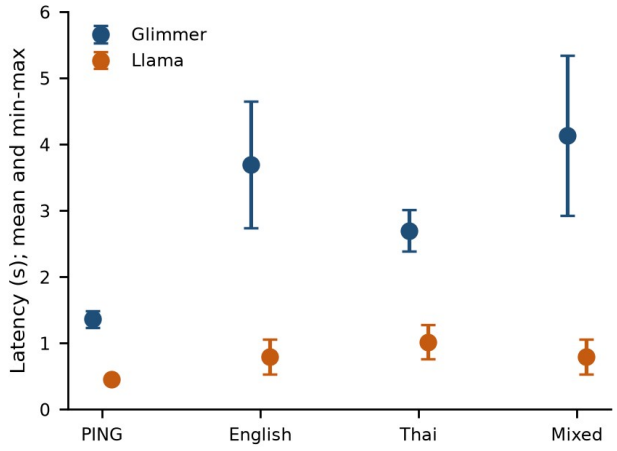


Fig. 3. Latency by prompt category in the principal comparison. Points are means of two requests; bars span the observed minimum and maximum, not SD or confidence intervals.

Prompt-stratified observations show a similar direction across all four categories. Glimmer's mean is 1.365 s for PING, 3.695 s for English, 2.701 s for Thai and 4.136 s for mixed text. Llama's corresponding means are 0.452, 0.794, 1.018 and 0.797 s. With only two examples per category, these values describe the supplied trace and do not support significance tests or a population-level ranking.

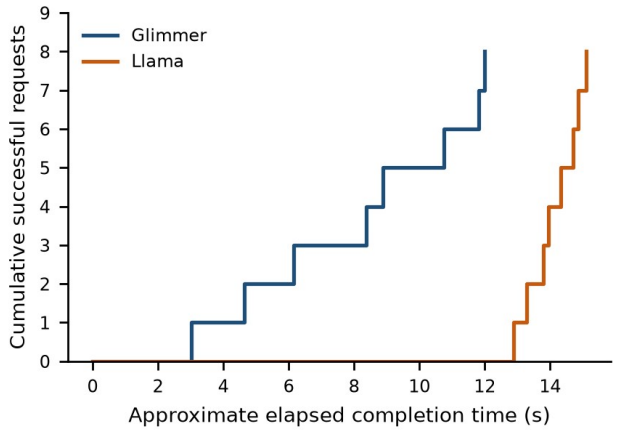


Fig. 4. Approximate cumulative completions from request timestamps plus recorded latency. The sequence illustrates the ordered model groups and shared concurrency; timestamps do not expose an independent provider-side queue.

The completion trace makes the ordered execution visible. The two endpoints are not driven by independent stationary arrival streams, and the later Llama requests encounter a different moment in the run. An external load generator, randomized/interleaved request order, repeated sessions and deployment metadata would be needed before attributing all observed differences to model architecture.

VI. Token Accounting and Output Budget

TABLE 5. PRINCIPAL RUN REPORTED USAGE TOTALS

Usage field	Glimmer	Llama
Prompt tokens	814	648
Completion tokens	3339	233
Reasoning subset	3063	0
Total tokens	4153	881
Other completion	276	233

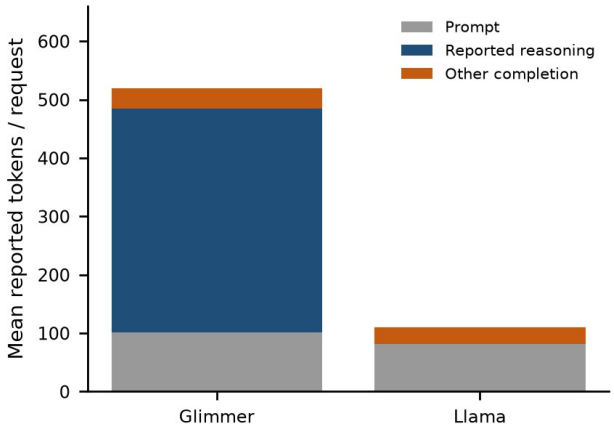


Fig. 5. Mean reported tokens per successful request. Reasoning is shown as a subset of completion tokens, with other completion tokens calculated by subtraction. A missing/zero reasoning field is not proof of an identical internal computation path.

Glimmer reports 3,063 reasoning tokens out of 3,339 completion tokens, or 91.7% of completion usage, in the principal run. Llama records zero reasoning tokens in the corresponding field. The remaining completion counts are 276 and 233 respectively. These are accounting fields, not character counts, visible-word counts or directly comparable computational work across tokenizers.

The first smoke run uses max_tokens=128 and yields an empty-or-unsupported-text error for Glimmer. At max_tokens=1024 in the next smoke run, Glimmer returns nonempty English text in 6,336.41 ms, reporting 745 completion tokens, of which 711 are reasoning tokens. Llama returns Thai text in 704.73 ms. The larger budget therefore accompanies a successful Glimmer response, but the failed response body, usage and finish reason were not retained.

A budget exhausted by reasoning is a plausible hypothesis for the 128-token failure, not a demonstrated cause. Other payload/content-handling issues remain possible. The records provide one failed low-budget observation and no controlled replication. Future instrumentation should retain sanitized finish reasons, provider request identifiers and complete usage for failures without exposing credentials or private text.

No prices, invoices, GPU billing durations or authenticated quota snapshots are included. This paper makes no monetary cost estimate. Dividing total usage tokens by wall time would mix prompt processing, reasoning, generated text and scheduling; it is not presented as model decoding tokens per second.

VII. Qualitative Text Inspection

Output inspection is limited to the retained excerpts, which are capped at 280 characters by the harness. There is no reference-answer file, blinded assessment protocol, language-normalization specification or independent evaluator. The following observations are examples, not an accuracy score or a claim that one model is categorically better at Thai.

The English examples from both models correct visible errors such as “Teh”, “meetng” and “shipmnt”, while retaining Mali, Somchai, Friday and tomorrow. Glimmer joins clauses with a comma; Llama varies punctuation and tense. These differences may be stylistic, and no scoring rubric in the supplied run makes them pass/fail conditions.

For the Thai examples in the principal run, Glimmer’s retained excerpts express the Friday deadline with the standard Friday spelling. Llama’s two excerpts preserve malformed deadline wording and, in one case, a malformed stock spelling. These visible defects matter for the office-note use case, but two observations do not measure a language-wide error rate. A future dataset should label dates and entity preservation separately from orthographic cleanup.

The mixed purchase-order excerpts retain PO#4471, 120 pieces, a shortage of two, the 48,500-baht amount and an undecided shipping date. Glimmer largely preserves the mixed-language form; Llama translates some Thai fragments into English. Whether that translation is desirable requires an explicit policy: a cleanup request alone does not define a target language or forbid translation.

For the PING prompt, Glimmer’s two excerpts are “PING” and Llama’s are “PING.”. The prompt wording includes punctuation around the requested response, and the harness checks nonempty text rather than exact-match compliance. The stored ok flag therefore cannot be reinterpreted as an exact-output or semantic-correctness score.

VIII. Long Context Probes and the Local Validation Boundary

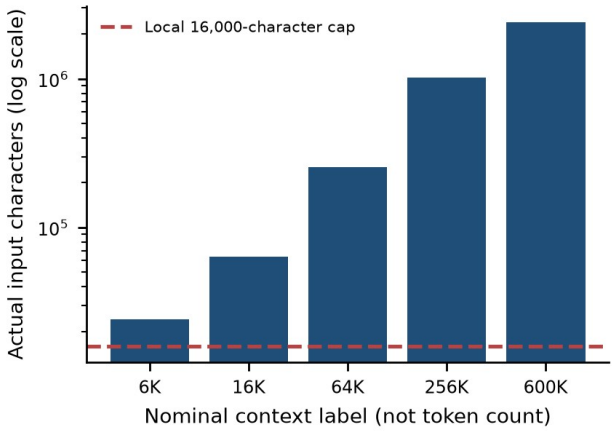


Fig. 6. Historical long-context input character lengths versus the then-active 16,000-character cap. The current adapter has a larger guard, but these old requests were not rerun and the new cohort uses only 848-character inputs.

All five nominal context inputs exceed 16,000 characters. Even the smallest is 24,064 characters, approximately 1.50 times the cap. The largest contains 2,400,012 characters. The historical validation path audited in v1.0 raised ValueError for inputs above the cap before request construction and urllib.request.urlopen [4]. All 50 live-configured context/load records contain that exact validator message.

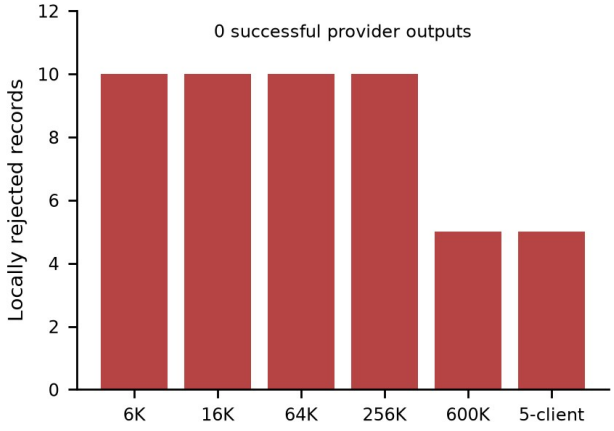


Fig. 7. Rejected records in each live-configured step. All errors share the local prompt-length validation message. These bars are not provider HTTP failures or context-window measurements.

These runs therefore establish a local application boundary. They do not establish that Fireworks accepted a 6K-labelled request, rejected a 600K-token request, completed long-context reasoning, or saturated a model endpoint. No HTTP response code, provider request ID, successful output or positive usage count is retained for these attempts. The generic live honesty string in their summaries overstates execution if read without the request-level errors.

Attempt completion rates in the stored summaries reach 406.17 records/s for one failed step. That rate describes rapid local validation/recording, not inference. The corrected successful live output rate for every such step is zero. Similarly, zero-valued percentile fields with n=0 denote an empty successful sample; they are missing data, not a zero-millisecond latency distribution.

The five-client step uses the same 24,064-character prompt and fails for the same reason. It adds five validation-error observations, not a live concurrency result. A model-specific tokenizer, validated provider limits, configurable application limits and verified network dispatch are prerequisites for a useful future context experiment. Merely increasing a label or deleting a guard would not establish a valid protocol.

IX. Simulation Is a Wiring Test

The six simulated runs produce 50 accepted synthetic records with no provider execution and zero reported tokens. Source inspection shows a deterministic assignment: latency_ms = 40 + ((37 × sequence) mod 120). The simulator returns that number immediately; it does not sleep for the assigned latency. Its numerical output is intentionally a stand-in for a live backend, not a wall-clock model measurement.

TABLE 6. SYNTHETIC LATENCY VALUES DESCRIBE THE GENERATOR

Requests/run	Mean ms	p50 ms	p95 ms
10	99.5	96	151
5	103.0	105	151

The ten-request runs share identical latency summaries across 6K, 16K, 64K and 256K labels because the formula depends on sequence, not input context. The two five-request runs share the second row above. Differences in their wall-clock record rates arise from fast local orchestration and recording, not from model speed or a long-context scaling law.

Assigned simulated latency and wall-clock simulation duration must not be combined as if they measured the same process. For example, several runs finish their full record set in roughly 0.02 s even though each assigned number is tens to hundreds of milliseconds. Reporting a “local model speedup” against real hosted inference would therefore be invalid. Simulation is useful for verifying schema, progress display and failure-free result plumbing only.

X. Resource Sampling and Measurement Coverage

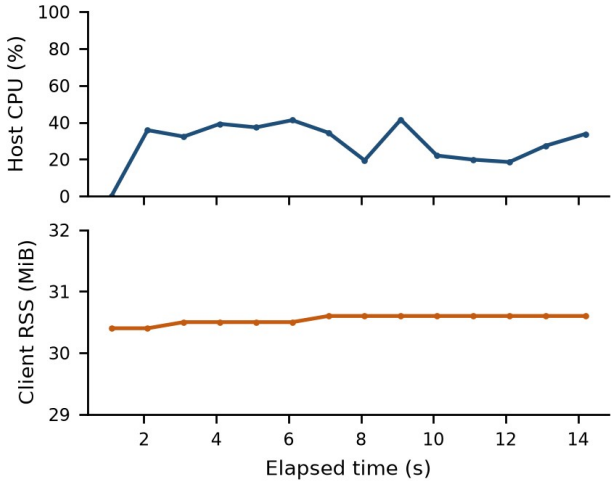


Fig. 8. Resource samples from the main live comparison. CPU is the client host's system-wide psutil.cpu_percent value; RSS is the benchmark process's resident memory divided by 1,048,576. Neither describes Fireworks GPU memory or utilization.

The main live run retains 14 samples, with process RSS between 30.4 and 30.6 MiB and sampled host CPU values between 0.0% and 41.5%. The first CPU value is an initial nonblocking psutil sample and should not be interpreted as an idle-period measurement. The code combines a system-wide CPU reading with a single-process RSS reading; the source resource_note alone does not communicate that distinction.

No resource samples are present in the 12 context/simulation steps. Their note says no samples (psutil not installed), but the current sampler can also return no sample when a very short run ends before its sampling interval. The retained records alone cannot distinguish package absence from timing or another sampling exception. Missing resource data are not converted into zero utilization.

The corpus contains no provider GPU model, VRAM consumption, power measurement, worker/coordinator container inventory, model-loading time or capacity allocation. The earlier Docker fleet benchmark is a different workload and is not merged into these inference results. No distributed multi-node model-execution claim follows from the present files.

XI. New Serverless Cohort and Recovered Protocol

Version 1.1 adds nine timestamped max-serverless exports recorded between 15:42 and 17:57 Bangkok time on 28 October 2026 [7]. They contain 22 distinct phase/run identifiers and 11,101 unique logical request records. These records are additional to, not replacements for, the 120 canonical requests in v1.0. The updated corpus therefore contains 37 canonical run/phase identifiers and 11,221 unique requests. Distinct run_id/sequence pairs were checked for duplication across the added files.

The new cohort returns 9,007 nonempty outputs and 2,094 errors. Of those errors, 1,841 have the detail "fireworks http 429" and 253 report empty or unsupported text. Adding the 19 historical comparison outputs gives 9,026 observed live texts across the complete corpus. Fifty historical local validation errors, one historical comparison error and 50 historical simulation records remain separate. A combined success percentage across these different workloads would have no useful operational interpretation.

TABLE 7. CHANGES FROM THE V1.0 EVIDENCE BOUNDARY

Measure	v1.0	v1.1
Canonical run/phase IDs	15	37
Unique logical requests	120	11221
New serverless phases	0	22
Observed live texts	19	9026
Historical simulations	50	50
Retained long-context live successes	0	0

Every added request has prompt_chars=848. The configuration's prompt_tokens=200 is a planning target, not a verified tokenizer count. Successful provider-reported prompt counts vary by route from 207 to 403 tokens. Input estimates in newer rows are characters divided by four (212). These short-input trials do not retest the 6K–600K nominal context probes and cannot establish long-context acceptance or quality.

The top-level topology label is "coordinator(local) -> worker routes -> clients". It describes logical orchestration, not a measured physical cluster: the files contain no Docker fleet inventory, coordinator-process count, worker-host allocation or provider GPU topology. Phase clients values are retained as requested ladder labels. A count of 1,000 logical requests or a ladder label of 1,000 does not independently prove 1,000 simultaneous network connections.

For each phase, this audit maps request rows by their run_id prefix, verifies request/success/error totals and successful token usage, and recomputes mean and nearest-rank p50/p95/p99 from successful latencies. Successful output rate is successes divided by retained phase elapsed_s. Those elapsed values are rounded to two decimals; recomputed rates can differ from a stored rate by about 0.01 requests/s. All reported rates in the new tables use this explicit denominator.

XII. Serverless Load Scaling and Conditional Operating Points

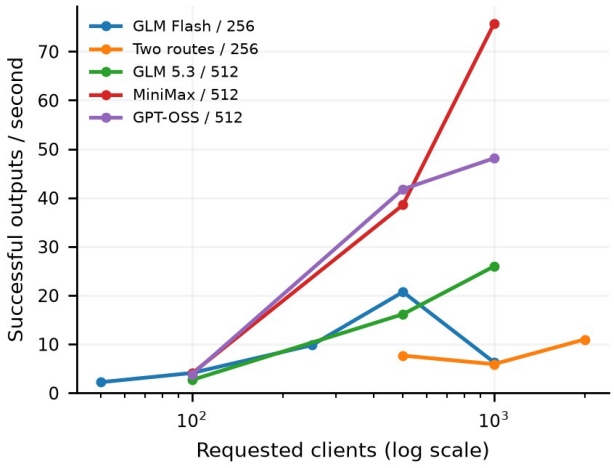


Fig. 9. Successful output rate versus requested client label for selected serverless ladders. The legend includes the output-token cap. Points are single phase observations, not replicated capacity estimates; workloads share short inputs but differ by model, cap and execution time.

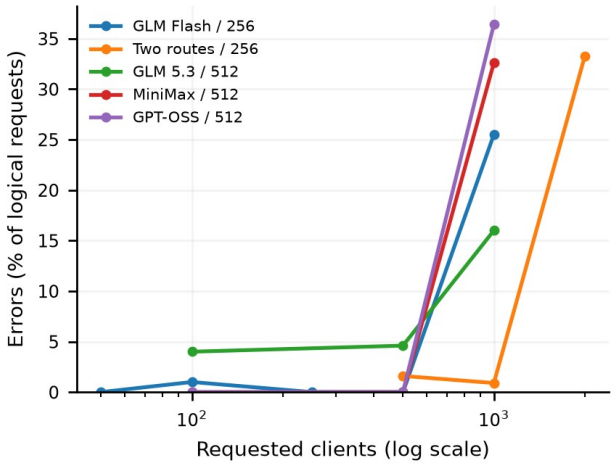


Fig. 10. Logical-request error fractions for the same ladders. Errors include HTTP 429 and empty/unsupported outputs. The 5% criterion used below is a descriptive selection rule, not a latency SLO or provider SLA.

The first GLM Flash single-route ladder records 50/50, 99/100, 250/250 and 500/500 successes at labels 50, 100, 250 and 500. At 500, successful output rate is approximately 20.70 requests/s and successful-request p95 is 22.24 s. At label 1,000, only 745/1,000 complete successfully: successful rate falls to 6.31 requests/s, while the stored attempt rate remains 8.46. The 255 errors comprise 254 HTTP 429 details and one empty/unsupported response.

Successful-only latency can improve numerically while service quality deteriorates. GLM Flash p95 falls from 22.24 s at label 500 to 2.62 s at 1,000, but the latter excludes 255 failed requests and includes a successful maximum exceeding 112 s. A lower conditional percentile in that overloaded phase is not evidence of an overall user-experience improvement. Failure rates, latency coverage and tail observations must accompany the percentile.

The accompanying three-route phase uses GLM Flash, DeepSeek Flash and Inkling at a nominal label of 500, but contains 501 actual logical requests: 167 per route. The audit preserves 501 rather than silently forcing the count to 500. It produces 429 successes and 72 empty/unsupported errors, for approximately 4.16 successful outputs/s. This phase does not show a throughput advantage from adding a third route.

The separate two-route ladder produces 492/500, 991/1,000 and 1,335/2,000 successes. Error fractions are 1.6%, 0.9% and 33.25%; successful rates are approximately 7.65, 5.90 and 10.98 requests/s. A label of 1,000 meets the error-only 5% criterion, but its successful p95 is 80.48 s. Consequently the highest passing client label is not the fastest useful operating point, and an error-only rule is insufficient for an interactive application.

For this report, a passing point means an observed phase with no more than 5% logical-request errors. Recomputing that rule yields highest tested passing labels of 500 for the single-route GLM Flash, GLM 5.3, MiniMax and GPT-OSS ladders, and 1,000 for the two-route ladder. These labels are contingent on the tested grid, account state, output caps and observed period. The exports' maxima strings must not be described as vendor maximum concurrency or a 429-only knee, because the stopping errors can also be empty/unsupported output failures.

TABLE 8. SELECTED OPERATING POINTS AND OVERLOAD OBSERVATIONS

Phase / cap	Label	OK/N	Goodput/s
GLM Flash /256	500	500/500	20.70
GLM Flash /256	1000	745/1000	6.31
Two routes /256	1000	991/1000	5.90
Two routes /256	2000	1335/2000	10.98
GLM 5.3 /512	500	477/500	16.14
GLM 5.3 /512	1000	840/1000	25.93
MiniMax /512	500	500/500	38.55
MiniMax /512	1000	674/1000	75.65
GPT-OSS /512	500	500/500	41.77
GPT-OSS /512	1000	636/1000	48.11

The MiniMax phase at label 1,000 is an example of why attempt rate can mislead: the stored value is 112.28 logical completions/s, but only 674 of 1,000 requests return text, yielding 75.65 successful outputs/s. GPT-OSS at the same label has stored attempt rate 75.66 but successful rate 48.11, with 364 HTTP 429 errors. A higher numerical rate with substantial rejection does not demonstrate a healthier service.

XIII. Model Screening and Output Budget Observations

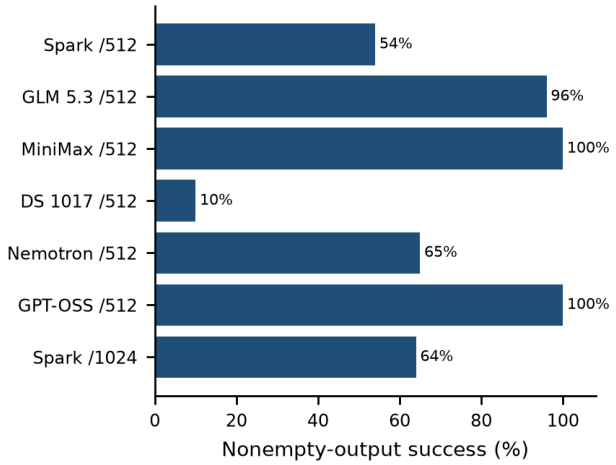


Fig. 11. Nonempty-output fractions in seven separate phases labelled 100 clients. Output caps appear after the slash. These are sequential screens under differing provider/account state, not a randomized same-time model ranking or quality score.

TABLE 9. SCREENS AT REQUESTED LABEL 100

Route / output cap	OK/N	Goodput/s	p95 s
Muse Spark Open / 512	54/100	1.27	36.88
GLM 5.3 /512	96/100	2.65	31.40
MiniMax /512	100/100	3.98	21.21
DeepSeek 1017 / 512	10/100	0.36	23.82
Nemotron /512	65/100	2.13	27.04
GPT-OSS /512	100/100	3.79	23.50
Muse Spark Open / 1024	64/100	1.34	39.14

GLM 5.3 progresses from 96/100 to 477/500 and 840/1,000 successful outputs. MiniMax and GPT-OSS have no observed errors at labels 100 and 500, but each develops substantial 429 rejection at 1,000. The DeepSeek variant ending in -1017 has 90 HTTP 429 failures out of 100 at its only retained phase. This is a routing/account-time observation, not proof that its model family intrinsically supports fewer clients.

Muse Spark Open at output cap 512 has 46 empty/unsupported-text errors out of 100. Its later cap-1024 phase has 64 successes, 32 HTTP 429 errors and four empty/unsupported-text errors. It would be incorrect to report this as a controlled demonstration that doubling the output cap fixed the earlier failures: time, rate-limit state and instrumentation changed, and the error mix shifted. A paired repeated budget experiment with finish reasons and failure usage is still needed.

The Nemotron-labelled phase has 65 nonempty texts and 35 empty/unsupported errors. The latter records set reached_provider=true but have no HTTP status and no retained usage. The current helper falls back to true for a generic RuntimeError; that flag alone is not affirmative transport evidence. Separate status/usage/request-ID evidence is preferable to accepting the helper's prose claim as proof.

Across the new cohort, 75 successful logical records declare attempts=2. This supplies a lower bound of 75 additional underlying attempts, so 11,101 logical rows must not be equated with exactly 11,101 HTTP calls. Failed rows do not consistently retain attempt counts, and no complete retry-level event log is supplied. Success/error rates and latency figures remain logical-request metrics; retry amplification and billing cannot be fully reconstructed.

XIV. Improved Error Timing and Remaining Evidence Gaps

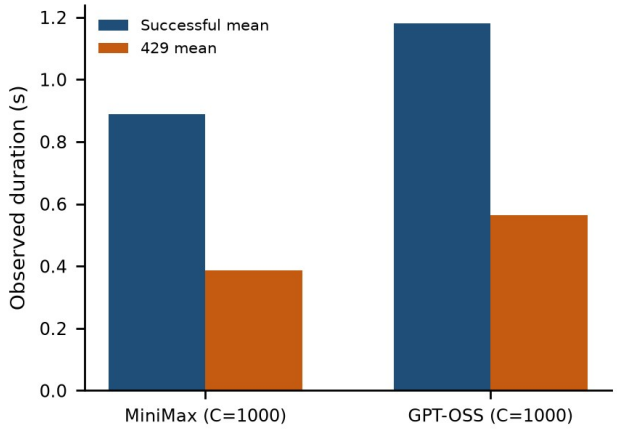


Fig. 12. Observed mean successful and HTTP 429 durations for MiniMax and GPT-OSS at label 1,000. The two phases contain 674/326 and 636/364 success/429 outcomes respectively. These means are conditional on outcome, not a combined user-latency distribution.

The last five new exports provide 3,500 rows with explicit http_status, reached_provider, usage_present and finish_reason keys. Of these, 3,461 have a non-null status: 2,649 are 200 and 812 are 429. All 3,500 set reached_provider=true, but 39 empty/unsupported records have neither a status nor positive usage. Furthermore, finish_reason is null in all 3,500 rows. Field presence therefore represents progress, but not complete evidence coverage.

The visible harness assigns status 200 to an accepted output in its success branch; it does not retain a raw response-header archive. Consequently these values are retained harness status indicators rather than an independently archived transport trace. Likewise, reached_provider=true on an unqualified RuntimeError is a heuristic. The revision keeps this distinction instead of treating every populated verification key as direct proof.

TABLE 10. COVERAGE IN THE 3,500 NEWER-SCHEMA ROWS

Evidence property	Rows	Interpretation
HTTP status key	3500	Schema field present
Non-null HTTP status	3461	2649 x 200; 812 x 429
Reached flag true	3500	39 lack status/usage
Non-null finish reason	0	Still unavailable

Error timing is now retained in these phases. At label 1,000, MiniMax's 429 mean is about 0.386 s, compared with its successful mean of 0.888 s. GPT-OSS's corresponding means are about 0.564 s and 1.180 s. By contrast, the DeepSeek-1017 phase's 429 mean is 20.26 s. Rate-limit rejection is not uniformly instantaneous, and older zero-valued error durations should remain unavailable historical measurements rather than being retroactively filled from newer runs.

The adapter visible during the v1.1 audit allows up to 4,194,304 prompt characters, replacing the historical 16,000-character cap. This is a code-level expansion, not a verified one-million-token capability: character-to-token ratios vary by model and language. The new 848-character requests do not test that larger boundary. The original 50 local long-context failures and their missing provider-capacity evidence remain unchanged.

XV. Reproducibility Boundaries and Financial Labels

Nine added exports include configuration ladders, phase summaries and request rows, but there are no corresponding new _work folders in the supplied result directory. The exact max-serverless driver was not found among the current app, bench and scripts files. Current RunConfig validation also limits requests_per_cell and total requests differently from some retained high-count phases. The paper can audit the exports internally, but cannot claim an exact replay of the historical load generator from the available source.

The stored billing string says prepaid-credits-only and no deployments; estimate_usd_worst values across the nine exports sum to USD 19.88. That is a sum of planning estimates covering configured ladders, some of which stopped early. It is neither an invoice nor measured spend nor a defensible upper bound on all charges: retry paths, failed-response usage, cache effects and historical price details are incomplete. The report introduces no current price assumptions and performs no metered action.

The observed source-level changes are material: positive error timing, explicit verification keys and a larger character guard are now visible. They address parts of the v1.0 instrumentation recommendations, while field nulls, helper heuristics, absent request IDs and missing driver provenance remain. These are cohort/schema differences supported by the retained fields and current source; the audit does not invent a run-time commit linking every phase to a particular implementation.

XVI. Threats to Validity

A. Sample size and ordering

The principal comparison has one session, eight requests per model and two observations per prompt. Smoke tests have one observation per model with different inputs. There is no repeated randomization, steady-state warmup protocol, power analysis or independent deployment replication. Error bars in the prompt plot are observed ranges, and no confidence intervals or significance claims are inferred from them.

B. Response success and quality

The ok flag denotes nonempty extracted content or a synthetic success. It does not require a factual, correctly spelled or instruction-compliant answer. Excerpts may omit trailing content; missing finish reasons prevent robust truncation analysis. Neither a scalar token total nor transport success is a task-quality evaluation.

C. Instrumentation and provenance

Historical exceptions overwrite latency with zero and discard most failure payloads. Newer-schema phases improve failure duration and verification fields, but finish reasons remain null and reached-provider classification can be heuristic. Summary labels still describe intended mode, stored totals.rps still counts failed completions, and some rates share one elapsed denominator. Missing historical source/driver and capacity metadata limit reproducibility and causal claims.

D. Scope of the supplied evidence

The files contain inference requests and simulated workload records, not training jobs. There is no SFT/DPO/RFT objective, training loss, evaluation checkpoint, held-out split, model weight update or deployment-change record. Fireworks training skills installed elsewhere in the project do not turn these results into fine-tuning evidence. Likewise, nominal context labels cannot substitute for tokenized inputs and dispatched provider requests.

XVII. Engineering Recommendations and Next Experiments

First, finish the partial instrumentation update documented in this revision. Keep configured mode, validation result, observed dispatch, actual HTTP status, provider request ID and useful output distinct. Replace generic RuntimeError-based provider-reach inference with affirmative transport evidence. Preserve failure duration and usage consistently, and investigate why finish_reason remains null rather than treating field presence as completion.

Second, make attempted-record rate and successful-model-output rate explicit in the schema and dashboards. Use unavailable/null latency statistics when n=0. Keep failed-attempt accounting, accepted responses and simulated records distinct, and ensure dashboards do not plot absent inference latency as zero or simulated assigned latency as observed duration.

Third, validate the expanded input guard with actual context-boundary experiments and a model-specific tokenizer. The present code raises the character cap, but the added requests are short. Confirm real dispatch, collect usage and evaluate a context-dependent answer before claiming long-context support. Set request/spending controls and an application latency/error objective before increasing offered load.

Fourth, interleave or randomize equal prompt sets across model targets, repeat sessions, capture endpoint resources and report both latency and task quality. Define a Thai cleanup rubric covering spelling, dates, names, numeric fields, missing information and language preservation. Keep a held-out set distinct from demonstration prompts. Record an explicit output-budget policy for reasoning models and retain sanitized finish reasons.

Finally, treat any fine-tuning proposal as a separate prospective study. Establish a validated baseline and a labelled dataset before selecting a training method. No new inference, deployment, training or billing action was performed to create this white paper; all recommendations here are future work, not completed experiments.

XVIII. Reproducibility and Evidence Index

The versioned analysis reads the supplied corpus, checks historical fingerprints, reconciles duplicate exports, maps the 22 new phases by run ID and recomputes counts, token totals and successful latency statistics. Derived outputs include analysis.json, phase-summary.csv, a canonical request dataset, twelve PNG/SVG figures and the complete v1.1 manuscript. Version 1.0 files are retained and checked for accidental changes.

```
python paper/fireworks-v1.1/analyze.py
python paper/fireworks-v1.1/manuscript.py
[Document-runtime Python] paper/fireworks-v1.1/build_docx.py
```

The first command re-audits evidence and generates figures, the second regenerates the manuscript structure, and the third clones the retained ResearchPaperTemplate. The final DOCX is rendered to PDF and inspected page by page. Rendering requires the documented Word/LibreOffice workflow and compatible Times fonts. Derived files are a report of existing evidence, not a request to rerun the metered comparison harness.

TABLE 11. HISTORICAL V1.0 RUN INVENTORY

Run identifier suffix	Class	N	OK
compare-144047	Deployment	2	1
compare-144454	Deployment	2	2
compare-144519	Deployment	16	16
6K context	Local error / sim	10 / 10	0 / 10
16K context	Local error / sim	10 / 10	0 / 10
64K context	Local error / sim	10 / 10	0 / 10
256K context	Local error / sim	10 / 10	0 / 10
600K context	Local error / sim	5 / 5	0 / 5
Five-client step	Local error / sim	5 / 5	0 / 5

In the inventory, comparison identifiers abbreviate the date prefix compare-20261028-. Each context row represents two separate runs sharing the context label: a live-configured local-error run and a simulated run. Full step identifiers, character lengths, mode, request counts and unrounded elapsed durations are retained in the machine-readable analysis [2].

TABLE 12. COMPLETE ADDED PHASE INVENTORY; RATES APPROXIMATE FROM ROUNDED PHASE DURATION

Phase	Label	OK/N	OK/s	p95 s
GLM Flash	50	50/50	2.21	20.50
GLM Flash	100	99/100	4.09	22.46
GLM Flash	250	250/250	9.80	22.32
GLM Flash	500	500/500	20.70	22.24
GLM Flash	1000	745/1000	6.31	2.62
3 routes	500	429/501	4.16	10.08
2 routes	500	492/500	7.65	42.32
2 routes	1000	991/1000	5.90	80.48
2 routes	2000	1335/2000	10.98	39.45
Spark512	100	54/100	1.27	36.88
GLM5.3	100	96/100	2.65	31.40
GLM5.3	500	477/500	16.14	17.58
GLM5.3	1000	840/1000	25.93	8.97
MiniMax	100	100/100	3.98	21.21
MiniMax	500	500/500	38.55	8.50
MiniMax	1000	674/1000	75.65	2.24
DS1017	100	10/100	0.36	23.82
Nemotron	100	65/100	2.13	27.04
GPT-OSS	100	100/100	3.79	23.50
GPT-OSS	500	500/500	41.77	8.09
GPT-OSS	1000	636/1000	48.11	2.23
Spark1024	100	64/100	1.34	39.14

XIX. Conclusion

The expanded corpus demonstrates nonempty serverless outputs under several short-input load ladders, along with substantial rate-limit and content-extraction failures. At label 500, MiniMax and GPT-OSS each complete 500/500 in their retained phases; at 1,000, rejection becomes material. Error-only passing labels, useful throughput, tail latency and response quality must be considered together. These single-phase observations do not establish vendor capacity, deployment size or a universal model ranking.

The historical long-context trials remain local-validation failures, despite the larger cap now visible in code. New error timing and metadata improve auditability but do not resolve missing finish reasons, heuristic reach flags, retry-level accounting or exact driver provenance. The next useful experiment is a repeated, token-verified, dispatch-verified load study with explicit latency/error objectives and task-quality scoring. No training, deployment or new inference was performed to create this revision.

References

[1] PASADHI project, supplied result corpus, test-result/firework/, 28 October 2026. Primary records: _work/<run-id>/{config.json,dataset.jsonl,run.log.jsonl,summary.json}; three root comparison exports are duplicate views.

[2] PASADHI retrospective evidence audit, bench/results/fireworks-report-v1.0/analysis.json and run-summary.csv, generated for this white paper. Includes input SHA-256 fingerprints, canonical-request counts and per-run checks.

[3] PASADHI load-test harness, app/hardtest.py. Historical interpretation and source hash retained in the v1.0 audit; updated source hash retained separately in the v1.1 audit. Run-time commits are unavailable.

[4] PASADHI Fireworks adapter, app/fireworks.py. Historical 16,000-character guard is evidenced by retained errors and v1.0 source audit; the v1.1-visible adapter uses a 4,194,304-character guard. Neither is a tokenizer-certified context limit.

[5] PASADHI deployment comparison driver, bench/compare_fireworks.py, current source inspected 28 October 2026. Defines the four prompt categories and timestamped export format.

[6] ResearchPaperTemplate, retained personal document template and visual layout specification. Used for two-column typesetting; not an official publisher acceptance or IEEE compliance certification.

[7] PASADHI, nine max-serverless JSON exports under test-result/firework/, filenames dated 2026-10-28 from 154234 through 175627. Twenty-two phase/run IDs and 11,101 logical request rows; no matching new _work folders supplied.

[8] PASADHI v1.1 retrospective audit, bench/results/fireworks-report-v1.1/analysis.json, phase-summary.csv and canonical-requests.jsonl. Stores full input fingerprints, phase metrics and current-source fingerprints.

[9] Current source inspection for this revision, app/hardtest.py and app/fireworks.py. The original max-serverless phase-driving implementation was not located; current source is not asserted to reproduce every historical phase.